

SIT741 - T09.D2 Project

Leon Wilson

2025-09-26

```
# All Libraries are loaded globally

# Read the data
accidents <- read_csv("car_accidents_victoria.csv")

## New names:
## Rows: 1644 Columns: 29
## -- Column specification
## ----- Delimiter: "," chr
## (29): ...1, EASTERN REGION, ...3, ...4, ...5, METROPOLITAN NORTH WEST RE...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...15`
## * `` -> `...16`
## * `` -> `...17`
## * `` -> `...19`
## * `` -> `...20`
## * `` -> `...21`
## * `` -> `...23`
## * `` -> `...24`
## * `` -> `...25`
## * `` -> `...27`
## * `` -> `...28`
## * `` -> `...29`
```

Q1: Car accident dataset

```
# Read in the two header rows
cav_data_link <- "car_accidents_victoria.csv"
top_row <- read_csv(cav_data_link, col_names = FALSE, n_max = 1)

## Rows: 1 Columns: 29
## -- Column specification -----
```

```

## Delimiter: ","
## chr (7): X2, X6, X10, X14, X18, X22, X26
## lgl (22): X1, X3, X4, X5, X7, X8, X9, X11, X12, X13, X15, X16, X17, X19, X20...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
second_row <- read_csv(cav_data_link, n_max = 1)

## New names:
## Rows: 1 Columns: 29
## -- Column specification
## ----- Delimiter: "," chr
## (29): ...1, EASTERN REGION, ...3, ...4, ...5, METROPOLITAN NORTH WEST RE...
## i Use `spec()` to retrieve the full column specification for this data. i
## Specify the column types or set `show_col_types = FALSE` to quiet this message.
## * `` -> `...1`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...15`
## * `` -> `...16`
## * `` -> `...17`
## * `` -> `...19`
## * `` -> `...20`
## * `` -> `...21`
## * `` -> `...23`
## * `` -> `...24`
## * `` -> `...25`
## * `` -> `...27`
## * `` -> `...28`
## * `` -> `...29`

# Make unique column names
column_names <- second_row %>%
  unlist(., use.names = FALSE) %>%
  make.unique(., sep = "__") # double underscore for uniqueness

column_names[2:5] <- str_c(column_names[2:5], "0", sep = "__")

# Read the main data, skipping the header rows
daily_accidents <- read_csv(cav_data_link, skip = 2, col_names = column_names)

## Rows: 1643 Columns: 29
## -- Column specification -----
## Delimiter: ","
## chr (1): DATE
## dbl (28): FATAL__0, SERIOUS__0, NOINJURY__0, OTHER__0, FATAL__1, SERIOUS__1,...
##

```

```
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Pivot the data to long format
tidy_accidents <- daily_accidents %>%
  pivot_longer(cols = -DATE, names_to = c("Accident_Type", "Region_index"),
    names_sep = "__", values_to = "Count")

# Map Region_index to actual region names
region_names <- as.character(top_row[1, -1]) # Drop the DATE column
regions <- unique(region_names[!is.na(region_names) & region_names != "NA" &
  region_names != ""])
tidy_accidents$Region <- regions[as.numeric(tidy_accidents$Region_index) +
  1] # +1 for R's 1-based indexing

# Remove the Region_index column
tidy_accidents <- tidy_accidents %>%
  dplyr::select(DATE, Region, Accident_Type, Count)

# Clean up types
tidy_accidents$DATE <- as.Date(tidy_accidents$DATE, format = "%d/%m/%Y")
tidy_accidents$Accident_Type <- as.factor(tidy_accidents$Accident_Type)
tidy_accidents$Region <- as.factor(tidy_accidents$Region)
tidy_accidents$Count <- as.integer(tidy_accidents$Count)

print(head(tidy_accidents))
```

```
## # A tibble: 6 x 4
##   DATE      Region      Accident_Type Count
##   <date>    <fct>      <fct>      <int>
## 1 2016-01-01 EASTERN REGION      FATAL          0
## 2 2016-01-01 EASTERN REGION      SERIOUS         1
## 3 2016-01-01 EASTERN REGION    NOINJURY         0
## 4 2016-01-01 EASTERN REGION      OTHER           5
## 5 2016-01-01 METROPOLITAN NORTH WEST REGION FATAL          0
## 6 2016-01-01 METROPOLITAN NORTH WEST REGION SERIOUS         7
```

##Q1.1

```
# number of rows before tidying
dim(accidents)
```

```
## [1] 1644 29
```

```
# number of rows after tidying
dim(tidy_accidents)
```

```
## [1] 46004 4
```

Q1.2

```
str(tidy_accidents)
```

```
## tibble [46,004 x 4] (S3: tbl_df/tbl/data.frame)
## $ DATE      : Date[1:46004], format: "2016-01-01" "2016-01-01" ...
## $ Region    : Factor w/ 7 levels "EASTERN REGION",...: 1 1 1 1 2 2 2 2 3 3 ...
```

```
## $ Accident_Type: Factor w/ 4 levels "FATAL","NOINJURY",...: 1 4 2 3 1 4 2 3 1 4 ...
## $ Count       : int [1:46004] 0 1 0 5 0 7 0 7 1 9 ...
```

```
# After tidying, the dataset has 46,004 rows and 4 columns. The data
# types for each column are: DATE: Date - stores calendar dates
# (e.g., '2016-01-01'). Region: Factor - categorical variable with 7
# levels (e.g., 'EASTERN REGION'). Accident_Type: Factor -
# categorical variable with 4 levels (e.g., 'FATAL', 'NOINJURY').
# Count: Integer - whole numbers representing accident counts.
```

```
# Data type selection tree: 1. Is the column made up of dates? - Yes
# → Date - No → Next step 2. Are values numbers? - Yes → Are they
# whole numbers? - Yes → Integer - No → Double (numeric with
# decimals) - No → Next step 3. Are values repeated
# labels/categories? - Yes → Factor (categorical) - No → Character
# (text)
```

```
# Explanation: - DATE is Date type, allowing time-based analysis. -
# Region and Accident_Type are Factor types, useful for grouping and
# modeling. - Count is Integer, representing the number of
# accidents.
```

Q1.3

```
# How many unique regions?
n_regions <- tidy_accidents %>%
  distinct(Region) %>%
  nrow()
print(paste("Number of regions:", n_regions))
```

```
## [1] "Number of regions: 7"
```

```
# What is the date range?
date_range <- range(tidy_accidents$DATE, na.rm = TRUE)
print(paste("Date range:", date_range[1], "to", date_range[2]))
```

```
## [1] "Date range: 2016-01-01 to 2020-06-30"
```

#Q1.4

```
# FATAL represents a collision in which at least one person is
# killed. This means that the crash caused at least one death.
# SERIOUS represents a collision in which at least one person is
# seriously injured but no one (other than a confirmed suicide) is
# killed. Serious injury is classified based on a defined list of 20
# injury types, including fractures, concussion, internal injuries,
# crushings, burns (excluding friction burns), severe cuts, and
# injuries causing death 30 or more days after the collision. The
# main difference: - FATAL collisions involve death of at least one
# person. - SERIOUS collisions involve severe injuries but no
# fatalities.
```

Q2: Tidy data

Q2.1 Cleaning up columns is already done in the above sections

Q2.2 Tidying data a)

```
# The daily accidents data frame meets the criteria for tidy data: 1.  
# Each variable has its own column. For example, accident types by  
# region are in separate columns: FATAL_0, SERIOUS_0, NOINJURY_0,  
# etc. 2. Each observation has its own row. Each row represents the  
# accident data for a specific date. 3. Each value has its own cell.  
# The number of accidents for each combination of date, region, and  
# accident type is stored in a separate cell.
```

b)

```
# b) Transforming the daily accidents data frame into tidy data I. 0  
# spreading (pivot_wider) operations are needed because the original  
# data is in wide format and only needs to be gathered. II. 1  
# gathering (pivot_longer) operation is needed to convert all  
# accident type-region columns into key-value pairs. III. Steps in  
# detail: 1. Use pivot_longer() to gather all columns except DATE  
# into two new columns: Accident_Type and Region_index. This turns  
# wide columns like FATAL__0, SERIOUS__1, etc. into long format with  
# one row per observation. 2. Map Region_index to actual region  
# names using a lookup vector. 3. Remove the Region_index column,  
# keeping only DATE, Region, Accident_Type, and Count. 4. Clean up  
# data types: convert DATE to Date, Accident_Type and Region to  
# factors, and Count to integer. IV. Print the head of the tidy  
# dataset:  
print(head(tidy_accidents))
```

```
## # A tibble: 6 x 4  
##   DATE      Region      Accident_Type Count  
##   <date>    <fct>      <fct>         <int>  
## 1 2016-01-01 EASTERN REGION FATAL          0  
## 2 2016-01-01 EASTERN REGION SERIOUS         1  
## 3 2016-01-01 EASTERN REGION NOINJURY        0  
## 4 2016-01-01 EASTERN REGION OTHER           5  
## 5 2016-01-01 METROPOLITAN NORTH WEST REGION FATAL          0  
## 6 2016-01-01 METROPOLITAN NORTH WEST REGION SERIOUS         7
```

c)

```
# Clean up types  
tidy_accidents$DATE <- as.Date(tidy_accidents$DATE, format = "%d/%m/%Y")  
tidy_accidents$Accident_Type <- as.factor(tidy_accidents$Accident_Type)  
tidy_accidents$Region <- as.factor(tidy_accidents$Region)  
tidy_accidents$Count <- as.integer(tidy_accidents$Count)  
  
print(head(tidy_accidents))
```

```
## # A tibble: 6 x 4  
##   DATE      Region      Accident_Type Count  
##   <date>    <fct>      <fct>         <int>
```

```
##   <date>      <fct>                <fct>      <int>
## 1 2016-01-01 EASTERN REGION          FATAL         0
## 2 2016-01-01 EASTERN REGION          SERIOUS        1
## 3 2016-01-01 EASTERN REGION          NOINJURY        0
## 4 2016-01-01 EASTERN REGION          OTHER          5
## 5 2016-01-01 METROPOLITAN NORTH WEST REGION FATAL         0
## 6 2016-01-01 METROPOLITAN NORTH WEST REGION SERIOUS        7
```

d)

```
tidy_accidents$Count[is.na(tidy_accidents$Count)] <- 0
# Check for missing values in each column
colSums(is.na(tidy_accidents))
```

```
##          DATE      Region Accident_Type      Count
##          0          0          0          0
```

Q3: Fitting distributions

Q3.1

```
total_accidents_df <- tidy_accidents %>%
  group_by(DATE, Region) %>%
  summarise(TOTAL_ACCIDENTS = sum(Count, na.rm = TRUE)) %>%
  ungroup()
```

```
## `summarise()` has grouped output by 'DATE'. You can override using the
## `.groups` argument.
```

```
# Fit Poisson
fit_poisson <- fitdist(total_accidents_df$TOTAL_ACCIDENTS, "pois")

# Fit Negative Binomial
fit_nbinom <- fitdist(total_accidents_df$TOTAL_ACCIDENTS, "nbinom")
library(fitdistrplus)

loglik_poisson <- fit_poisson$loglik
loglik_nbinom <- fit_nbinom$loglik

cat("Poisson log-likelihood:", loglik_poisson, "\n")
```

```
## Poisson log-likelihood: -49698.42
```

```
cat("Negative binomial log-likelihood:", loglik_nbinom, "\n")
```

```
## Negative binomial log-likelihood: -32541.13
```

```
# Q3.2 Poisson log-likelihood: -49698.42 Negative binomial
# log-likelihood: -32541.13 The negative binomial distribution fits
# the data better because it has a higher (less negative)
# log-likelihood value. This indicates the model describes the data
# more accurately. The negative binomial model accounts for
# overdispersion (variance greater than mean) in count data, which
# the Poisson model cannot handle as it assumes mean equals variance.
# Therefore, the negative binomial distribution is more flexible and
# appropriate for modeling accident counts, resulting in a better fit.
```

Q3.3

```
# types to analyze
types_to_fit <- c("FATAL", "SERIOUS")

results <- data.frame(Distribution = character(), Accident_Type = character(),
  LogLikelihood = numeric(), stringsAsFactors = FALSE)

for (atype in types_to_fit) {
  # Subset data for accident type
  type_data <- tidy_accidents %>%
    filter(Accident_Type == atype)
  type_counts <- round(type_data$Count) # Make sure counts are integers

  # Fit distributions
  fit_pois <- fitdist(type_counts, "pois")
  fit_nb <- fitdist(type_counts, "nbinom")
  fit_geom <- fitdist(type_counts, "geom")

  # Store results
  results <- rbind(results, data.frame(Distribution = "Poisson", Accident_Type = atype,
    LogLikelihood = fit_pois$loglik), data.frame(Distribution = "Negative Binomial",
    Accident_Type = atype, LogLikelihood = fit_nb$loglik), data.frame(Distribution = "Geometric",
    Accident_Type = atype, LogLikelihood = fit_geom$loglik))
}

print(results)
```

##	Distribution	Accident_Type	LogLikelihood
## 1	Poisson	FATAL	-4428.277
## 2	Negative Binomial	FATAL	-4426.347
## 3	Geometric	FATAL	-4439.173
## 4	Poisson	SERIOUS	-26708.205
## 5	Negative Binomial	SERIOUS	-22092.466
## 6	Geometric	SERIOUS	-22094.669

Analysis: In fitting Poisson, Negative Binomial, and Geometric distributions to FATAL and SERIOUS accident counts, we compare log-likelihoods where higher (less negative) indicates better fit. For FATAL accidents, the Negative Binomial has the highest log-likelihood (-4426.347), slightly better than Poisson (-4428.277) and Geometric (-4439.173). This suggests Negative Binomial better models count variability. For SERIOUS accidents, Negative Binomial significantly outperforms Poisson (-22092.466 vs -26708.205), and is marginally better than Geometric (-22094.669), indicating greater overdispersion that Poisson cannot capture. Overall, the Negative Binomial distribution best fits the accident data, especially for SERIOUS accident counts where data variance is larger than mean. This makes it more reliable for statistical inference and prediction in accident modeling.

Q4: Source weather data

Q4.1

```
# I plan to use weather data from the Bureau of Meteorology (BOM)  
# historical weather observations and statistics. This data source  
# is chosen because it is the official government provider of  
# Australian weather data, ensuring high-quality, accurate, and  
# comprehensive meteorological records. BOM data includes daily  
# temperature, precipitation, and other relevant weather variables  
# across multiple locations and time periods, aligning well with the  
# accident dataset's temporal and spatial coverage. The use of BOM  
# data allows for consistent and reliable analysis of how weather  
# events may correlate with road accidents. Its public  
# accessibility, extensive documentation, and authoritative status  
# make it ideal for this study.
```

Q4.2

```
# BOM format: X3 = Year, X4 = Month, X5 = Day, X6 = Value  
read_bom_value <- function(filepath) {  
  df_raw <- read_csv(filepath, col_names = FALSE, show_col_types = FALSE)  
  
  df_vals <- df_raw %>%  
    transmute(date = make_date(as.integer(X3), as.integer(X4), as.integer(X5)),  
              value = as.numeric(X6)) %>%  
    filter(!is.na(date))  
  
  return(df_vals)  
}
```

```
stations <- c("mel", "sco", "ben", "wan", "sale", "hor", "war")  
data_dir <- "C:/Users/Leon/Desktop/Study Materials/T4/SIT741 - Statistical Data Analysis/Task/Task D"  
for (st in stations) {  
  min_file <- file.path(data_dir, paste0("min_", st, ".csv"))  
  max_file <- file.path(data_dir, paste0("max_", st, ".csv"))  
  rain_file <- file.path(data_dir, paste0("rain_", st, ".csv"))  
  
  min_df <- read_bom_value(min_file) %>%  
    rename(min_temp = value)  
  max_df <- read_bom_value(max_file) %>%  
    rename(max_temp = value)  
  rain_df <- read_bom_value(rain_file) %>%  
    rename(rain = value)  
  
  # merge by date  
  station_df <- reduce(list(max_df, min_df, rain_df), left_join, by = "date") %>%  
    dplyr::select(date, max_temp, min_temp, rain) %>%  
    filter(date >= as.Date("2016-01-01"), date <= as.Date("2020-06-30"))  
  
  assign(paste0("df_", st), station_df, envir = .GlobalEnv)  
}
```

```
## Warning: There were 4 warnings in `transmute()`.
```

```
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
```

```
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.  
## Caused by warning in `vapply()`:  
## ! NAs introduced by coercion  
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.  
## Caused by warning in `vapply()`:  
## ! NAs introduced by coercion  
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.  
## Caused by warning in `vapply()`:  
## ! NAs introduced by coercion  
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.  
## Caused by warning in `vapply()`:  
## ! NAs introduced by coercion  
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.  
## Caused by warning in `vapply()`:  
## ! NAs introduced by coercion  
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.  
## Caused by warning in `vapply()`:  
## ! NAs introduced by coercion  
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.  
## There were 4 warnings in `transmute()`.  
## The first warning was:  
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),  
##   as.integer(X5))`.
```

```
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
## There were 4 warnings in `transmute()`.
## The first warning was:
## i In argument: `date = make_date(as.integer(X3), as.integer(X4),
##   as.integer(X5))`.
## Caused by warning in `vapply()`:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
```

Q4.3

```
# How many rows are in the local weather data?
num_rows <- nrow(df_mel)
print(paste("Number of rows:", num_rows))
```

```
## [1] "Number of rows: 1643"
```

```
# What time period does the data cover?
start_date <- min(df_mel$date)
end_date <- max(df_mel$date)
print(paste("Time period covered: from", start_date, "to", end_date))
```

```
## [1] "Time period covered: from 2016-01-01 to 2020-06-30"
```

Q5 Heatwaves, precipitation and road traffic accidents

Q5.1

```
# The Excess Heat Factor (EHF) measures heatwave intensity relative  
# to local climate. It combines two indices based on 3-day-averaged  
# daily mean temperatures: 1. EHIsig compares the 3-day average to  
# the 95th percentile of historic local temps - indicates unusual  
# heat. 2. EHIaccl compares the 3-day average to the prior 30 days -  
# reflects recent acclimatisation. EHF = EHIsig * max(1, EHIaccl),  
# ensuring positive EHF only when EHIsig is positive. A heatwave is  
# defined as 3+ consecutive days with positive EHF. This captures  
# both climatic context and short-term acclimatisation effects.
```

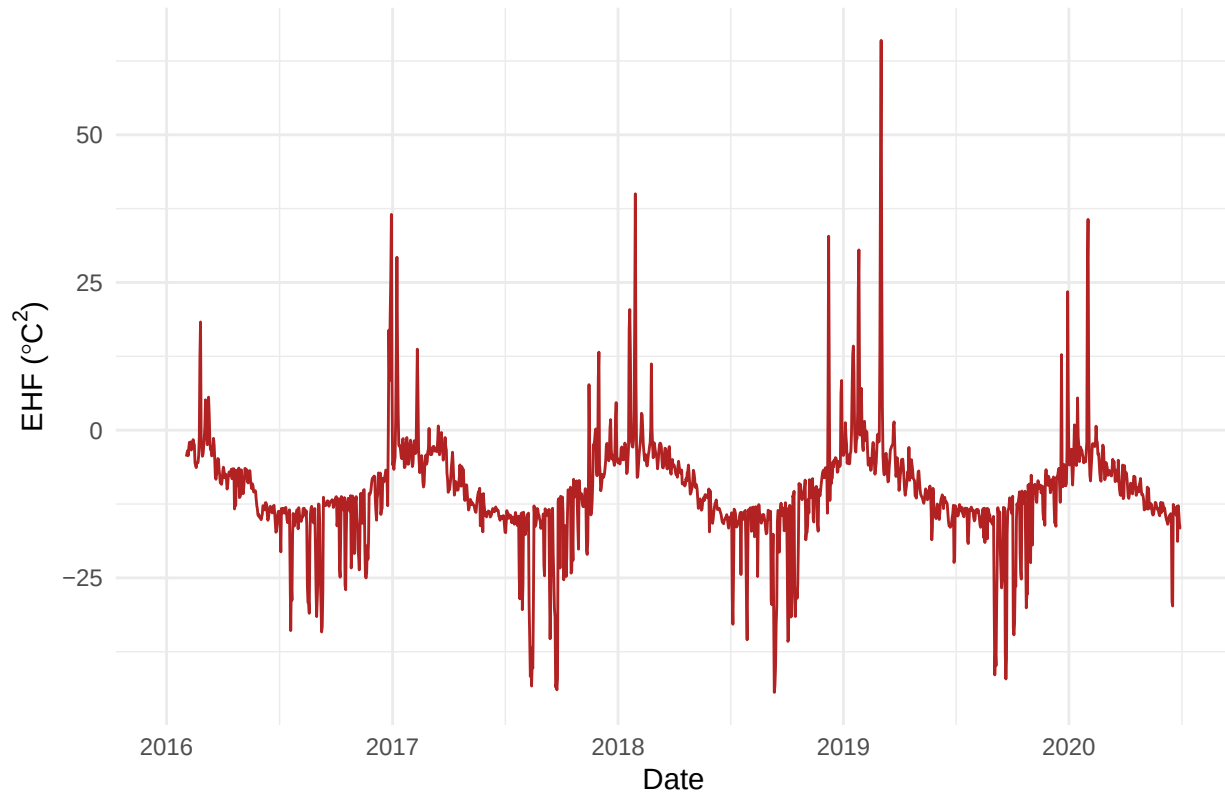
Q5.2

```
# Calculate Daily Mean Temperature  
df_mel <- df_mel %>%  
  mutate(DMT = (max_temp + min_temp)/2)  
  
# Calculate 3-day running mean (DMT_3d)  
df_mel <- df_mel %>%  
  arrange(date) %>%  
  mutate(DMT_3d = rollapply(DMT, width = 3, align = "right", fill = NA,  
    FUN = mean, na.rm = FALSE))  
  
# Calculate 30-day mean prior to 3-day period (DMT_30d_bg)  
get_30d_bg <- function(x) {  
  n <- length(x)  
  out <- rep(NA_real_, n)  
  for (i in seq_len(n)) {  
    if (i > 32) {  
      out[i] <- mean(x[(i - 32):(i - 3)], na.rm = FALSE)  
    }  
  }  
  out  
}  
df_mel$DMT_30d_bg <- get_30d_bg(df_mel$DMT)  
  
# Calculate T95 (approximation: 95th percentile of 3-day means)  
T95 <- quantile(df_mel$DMT_3d, 0.95, na.rm = TRUE)  
  
# Calculate EHIsig and EHIaccl  
df_mel <- df_mel %>%  
  mutate(EHIsig = DMT_3d - T95, EHIaccl = DMT_3d - DMT_30d_bg) %>%  
  # Now calculate EHF in a second mutate  
  mutate(EHF = EHIsig * pmax(1, EHIaccl))  
  
# Plot the daily EHF values  
ggplot(df_mel, aes(x = date, y = EHF)) + geom_line(color = "firebrick") +  
  labs(title = "Daily Excess Heat Factor (EHF) at Melbourne Airport",  
    x = "Date", y = expression("EHF (" * degree * C^2 * ")")) + theme_minimal()
```

```
## Warning: Removed 32 rows containing missing values or values outside the scale range
```

```
## (`geom_line()`).
```

Daily Excess Heat Factor (EHF) at Melbourne Airport



Q6. Model planning

#Q6.1. Model planning

*# a) Main goal: Predict daily road accident counts in the
Metropolitan North West Region using weather variables (max_temp,
min_temp, rain, temp_range, lagged rain) and heatwave intensity
(EHF). b) Relevance: Supports emergency services by forecasting
high-risk days, allowing better staffing and resource allocation.
c) Potential users: Emergency service planners, traffic
authorities, public safety officials, and local government road
safety teams.*

Q6.2. Relationship and data

*# a) Relationship: Model total daily accidents as a function of
current weather, heatwave intensity, and delayed rainfall effects
(lagged rain), as well as temperature variability (temp_range). b)
Response variable: total_accidents (daily accident counts) c)
Predictors: max_temp, min_temp, rain, EHF, lagged rain (rain_lag1),
temp_range d) Availability: Variables are routinely collected by
meteorological and transport agencies, and available shortly after
observation. e) Future data characteristics: Historical patterns
and measurement systems are consistent, supporting model*

```
# generalizability.
```

Q6.3.

```
# Linear regression (LM) and Generalized Additive Models (GAMs) are
# used. GAMs with Negative Binomial family capture nonlinear effects
# of weather and overdispersion in accident counts. Model fit is
# evaluated using AIC, residual diagnostics, and inclusion of lagged
# or derived predictors (e.g., lagged rain, temp_range) to improve
# predictive accuracy.
```

Q7: Model the number of road traffic accidents

```
stations <- c("mel", "sco", "ben", "wan", "sale", "hor", "war")
for (st in stations) {
  df <- get(paste0("df_", st))
  cat("Station:", st, "\n")
  print(colSums(is.na(df)))
  cat("Rows with any missing:", sum(!complete.cases(df)), "\n\n")
}
```

```
## Station: mel
##      date max_temp min_temp      rain      DMT      DMT_3d DMT_30d_bg
##        0         0         0         0         0         2         32
##      EHIsig      EHlacc1      EHF
##        2         32         32
## Rows with any missing: 32
##
## Station: sco
##      date max_temp min_temp      rain
##        0         25         34         11
## Rows with any missing: 58
##
## Station: ben
##      date max_temp min_temp      rain
##        0         23         18         25
## Rows with any missing: 34
##
## Station: wan
##      date max_temp min_temp      rain
##        0          9          0          7
## Rows with any missing: 16
##
## Station: sale
##      date max_temp min_temp      rain
##        0          1          1         13
## Rows with any missing: 14
##
## Station: hor
##      date max_temp min_temp      rain
##        0         14          9         21
## Rows with any missing: 28
##
```

```
## Station: war
##      date max_temp min_temp      rain
##      0       2       17         6
## Rows with any missing: 24
```

Q7.1

```
# I selected the Metropolitan North West Region for this analysis.
# This region was chosen because it was the only region in the
# dataset with complete data and no missing values for the relevant
# variables. T
```

```
acc_nw_daily <- tidy_accidents %>%
  filter(Region == "METROPOLITAN NORTH WEST REGION") %>%
  group_by(DATE) %>%
  summarise(total_accidents = sum(Count, na.rm = TRUE)) %>%
  ungroup()
```

Q7.2

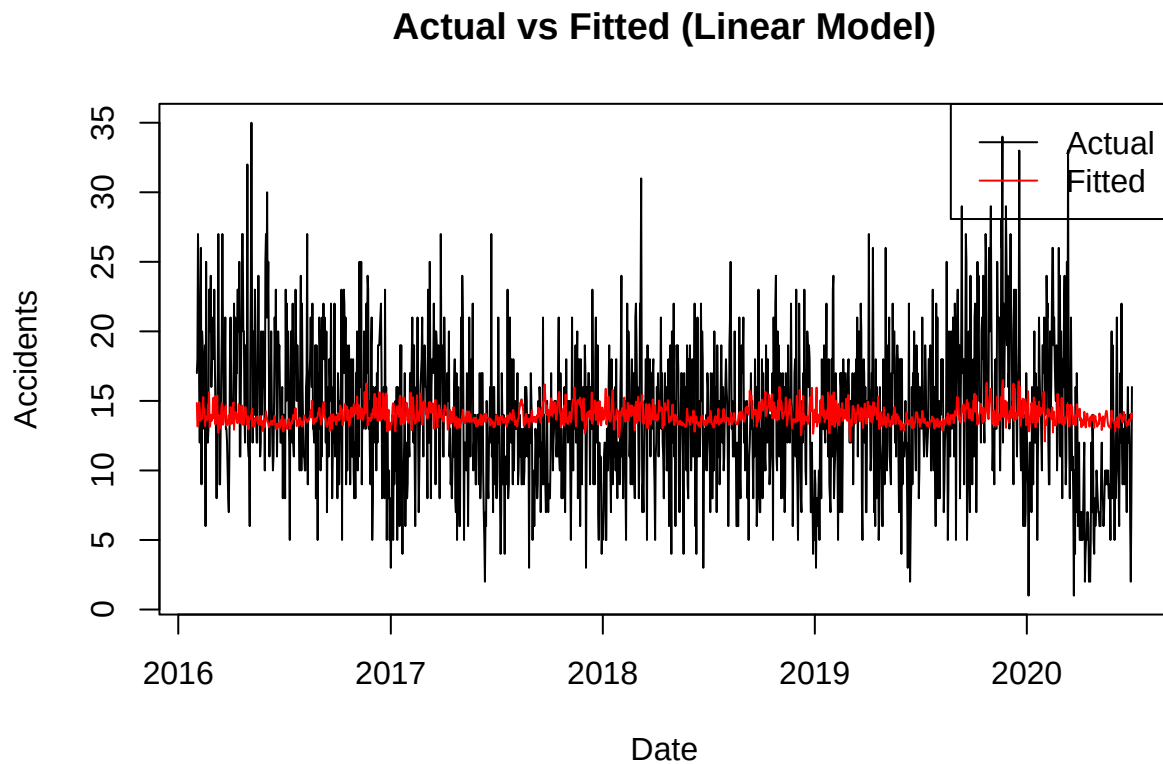
```
# Merge accident and weather data by date
model_df <- left_join(acc_nw_daily, df_mel, by = c(DATE = "date"))

# Keep only complete rows
model_df <- model_df %>%
  filter(!is.na(total_accidents), !is.na(max_temp), !is.na(min_temp),
         !is.na(rain), !is.na(EHF))

# Fit linear model
fit_lm <- lm(total_accidents ~ max_temp + min_temp + rain + EHF, data = model_df)
summary(fit_lm)
```

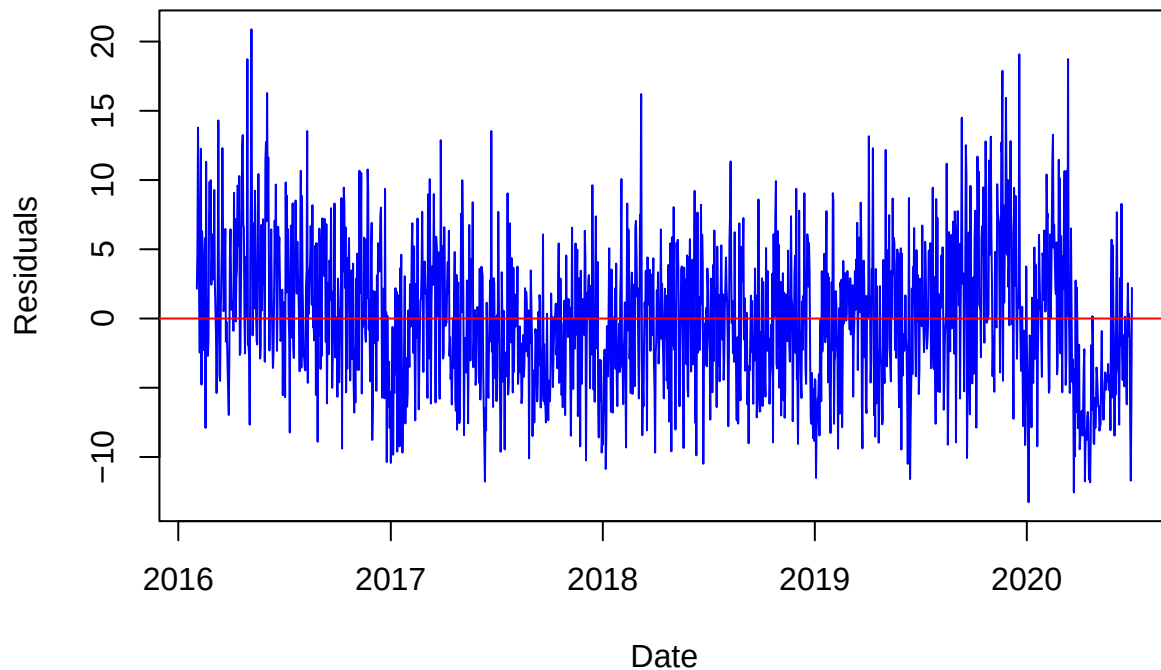
```
##
## Call:
## lm(formula = total_accidents ~ max_temp + min_temp + rain + EHF,
##     data = model_df)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.2678  -3.6379  -0.2568   3.3358  20.8818
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 11.538189   0.632953  18.229  < 2e-16 ***
## max_temp     0.133481   0.028280   4.720 2.56e-06 ***
## min_temp    -0.080688   0.041731  -1.934  0.0533 .
## rain         0.003727   0.027704   0.135  0.8930
## EHF        -0.040619   0.018269  -2.223  0.0263 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.171 on 1606 degrees of freedom
## Multiple R-squared:  0.01519,    Adjusted R-squared:  0.01274
## F-statistic: 6.195 on 4 and 1606 DF,  p-value: 6.042e-05
```

```
# Plot actual vs fitted
plot(model_df$DATE, model_df$total_accidents, type = "l", col = "black",
      ylab = "Accidents", xlab = "Date", main = "Actual vs Fitted (Linear Model)")
lines(model_df$DATE, fitted(fit_lm), col = "red")
legend("topright", legend = c("Actual", "Fitted"), col = c("black", "red"),
      lty = 1)
```



```
# Residuals over time
plot(model_df$DATE, resid(fit_lm), type = "l", col = "blue", ylab = "Residuals",
      xlab = "Date", main = "Residuals Over Time (Linear Model)")
abline(h = 0, col = "red")
```

Residuals Over Time (Linear Model)



The linear model is likely insufficient for modeling the trend of total accidents. 1. The 'Actual vs Fitted' plot shows the fitted line (red) remains nearly flat and does not capture the sharp peaks or fluctuations present in the actual accident count (black). 2. The residuals plot shows residuals are not randomly scattered around zero, but rather exhibit persistent patterns, indicating the model does not fully capture the data structure. 3. The low R-squared (0.015) from the summary confirms the linear model explains very little of the variance in accident counts. These observations suggest that the relationship between weather variables and accidents is likely nonlinear

##Q7.3

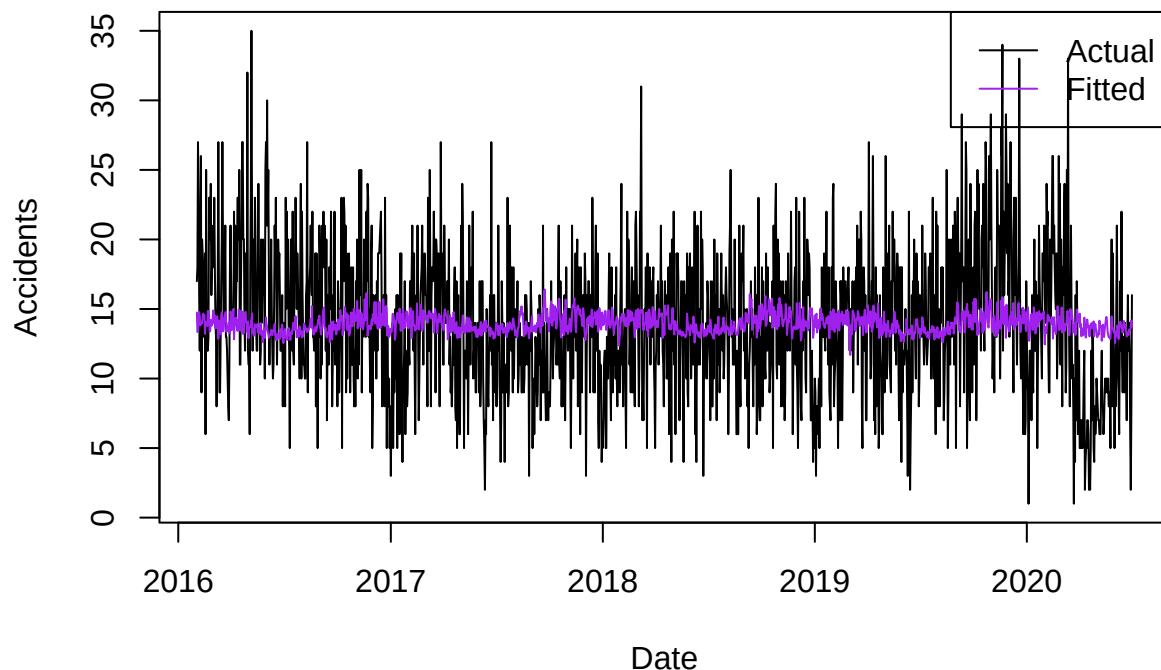
```
# Fit GAM (Negative Binomial family handles overdispersion)
fit_gam <- gam(total_accidents ~ s(max_temp) + s(min_temp) + s(rain) +
  s(EHF), data = model_df, family = nb())
summary(fit_gam)
```

```
##
## Family: Negative Binomial(14.608)
## Link function: log
##
## Formula:
## total_accidents ~ s(max_temp) + s(min_temp) + s(rain) + s(EHF)
##
## Parametric coefficients:
```

```
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept) 2.633790    0.009333   282.2  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##           edf Ref.df Chi.sq  p-value
## s(max_temp) 2.299  2.915 24.572 1.93e-05 ***
## s(min_temp) 1.394  1.699  3.469  0.0882 .
## s(rain)      1.002  1.005  0.041  0.8452
## s(EHF)       1.009  1.017  4.717  0.0309 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.0157   Deviance explained = 1.77%
## -REML = 4931.1   Scale est. = 1           n = 1611
```

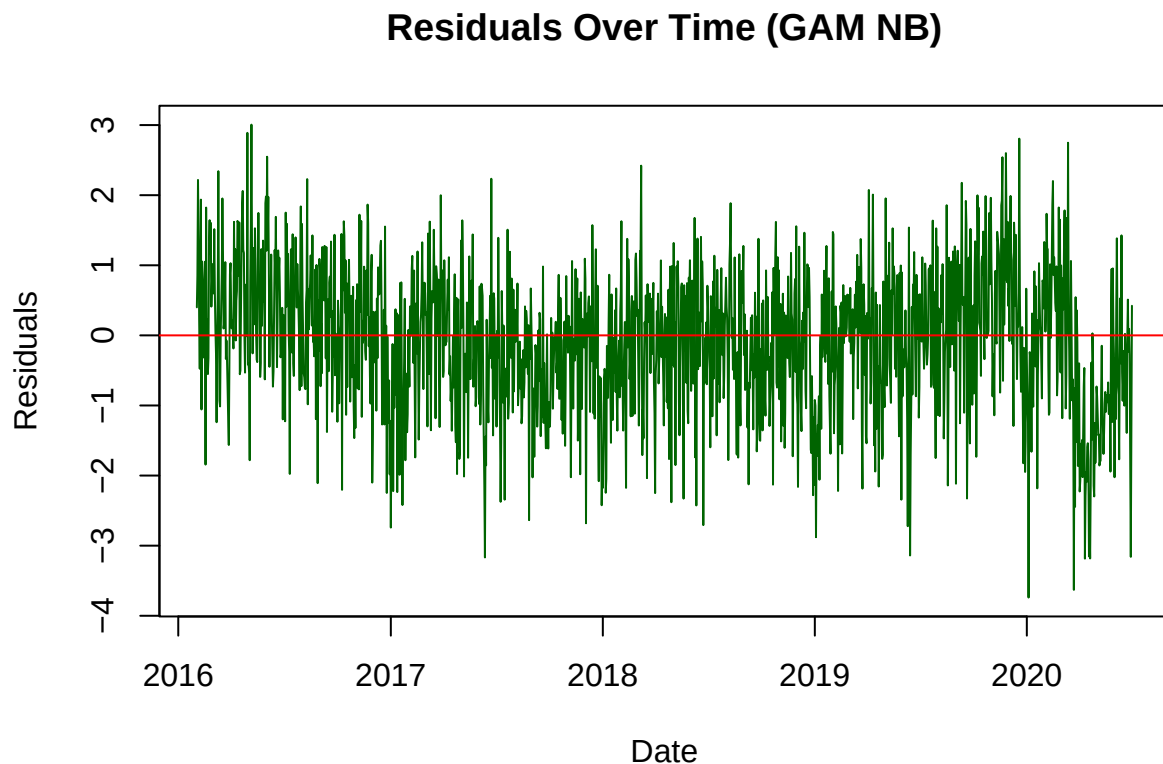
```
# Plot actual vs fitted
plot(model_df$DATE, model_df$total_accidents, type = "l", col = "black",
      ylab = "Accidents", xlab = "Date", main = "Actual vs Fitted (GAM NB)")
lines(model_df$DATE, fitted(fit_gam), col = "purple")
legend("topright", legend = c("Actual", "Fitted"), col = c("black", "purple"),
      lty = 1)
```

Actual vs Fitted (GAM NB)



```
# Residuals over time
plot(model_df$DATE, resid(fit_gam), type = "l", col = "darkgreen", ylab = "Residuals",
      xlab = "Date", main = "Residuals Over Time (GAM NB)")
```

```
abline(h = 0, col = "red")
```



Q7.3: Assessment of GAM model fit The GAM relaxes the linearity assumption and models nonlinear effects of weather predictors on daily accident counts. The 'Actual vs Fitted (GAM NB)' plot shows the fitted values (purple line) capture the general level of accidents but fail to reproduce all peaks and variability in the data. The residuals over time fluctuate around zero without large systematic deviations, but some short-term clustering is still present, suggesting unexplained structure remains. From the model summary: - Only max_temp (p = 1.93e-05) and EHF (p = 0.0309) are significant predictors. - min_temp is marginally significant (p = 0.0882), rain is not significant (p = 0.8452). - Adjusted R-squared is very low (0.0157), explaining only ~1.77% of deviance. Overall, while the GAM captures nonlinear relationships and slightly improves model flexibility over a linear model, residual patterns and low explained variance indicate that additional predictors or transformations may be needed for better model performance.

Q7.4

```
# Poisson GLM
fit_pois <- glm(total_accidents ~ max_temp + min_temp + rain + EHF, data = model_df,
  family = poisson())
```

```
# Negative Binomial GLM
fit_nb <- glm.nb(total_accidents ~ max_temp + min_temp + rain + EHF, data = model_df)
```

```
# Compare AIC
aic_results <- data.frame(Model = c("Linear LM", "Poisson GLM", "NegBin GLM",
  "GAM NegBin"), AIC = c(AIC(fit_lm), AIC(fit_pois), AIC(fit_nb), AIC(fit_gam)))
aic_results
```

```
##           Model           AIC
## 1  Linear LM  9873.070
## 2 Poisson GLM 10270.169
## 3 NegBin GLM  9839.608
## 4  GAM NegBin  9838.868
```

```
summary(fit_pois)
```

```
##
## Call:
## glm(formula = total_accidents ~ max_temp + min_temp + rain +
##      EHF, family = poisson(), data = model_df)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  2.4653952  0.0324514  75.972 < 2e-16 ***
## max_temp      0.0093819  0.0014382   6.523 6.88e-11 ***
## min_temp     -0.0057192  0.0021517  -2.658  0.00786 **
## rain          0.0002243  0.0014609   0.154  0.87795
## EHF          -0.0028607  0.0009354  -3.058  0.00223 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##      Null deviance: 3207.4  on 1610  degrees of freedom
## Residual deviance: 3160.5  on 1606  degrees of freedom
## AIC: 10270
##
## Number of Fisher Scoring iterations: 4
```

```
summary(fit_nb)
```

```
##
## Call:
## glm.nb(formula = total_accidents ~ max_temp + min_temp + rain +
##      EHF, data = model_df, init.theta = 14.61745574, link = log)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  2.462255  0.045601  53.996 < 2e-16 ***
## max_temp      0.009384  0.002029   4.625 3.75e-06 ***
## min_temp     -0.005487  0.003015  -1.820  0.0687 .
## rain          0.000216  0.002025   0.107  0.9151
## EHF          -0.002937  0.001315  -2.233  0.0256 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## (Dispersion parameter for Negative Binomial(14.6175) family taken to be 1)
##
##      Null deviance: 1691  on 1610  degrees of freedom
## Residual deviance: 1667  on 1606  degrees of freedom
## AIC: 9839.6
##
## Number of Fisher Scoring iterations: 1
##
##              Theta: 14.62
##             Std. Err.: 1.08
##
## 2 x log-likelihood: -9827.608
```

Model Comparison and Interpretation

```
# AIC values for all models: - Linear LM: 9873.070 - Poisson GLM:
# 10270.169 - Negative Binomial GLM: 9839.608 - GAM NegBin: 9838.868
# The GAM with Negative Binomial family has the lowest AIC,
# indicating the best fit among all models. Negative Binomial GLM
# Coefficients Intercept (2.462): Highly significant baseline
# log-accident count. max_temp (0.00938): Significant positive
# effect; higher max temperatures slightly increase daily accidents.
# min_temp (-0.00549): Marginally significant negative effect (p =
# 0.0687); higher minimum temperatures slightly decrease accidents.
# rain (0.000216): Not significant; rainfall does not have a
# detectable effect in this model. EHF (-0.00294): Significant
# negative effect; extreme heat factor slightly reduces daily
# accident counts.
```

```
# Model Fit Notes: - Residual deviance: 1667 on 1606 df (vs null
# deviance 1691 on 1610 df) shows moderate improvement over the null
# model. - Theta = 14.62 indicates moderate overdispersion,
# justifying the Negative Binomial approach over Poisson. - AIC =
# 9839.6 confirms better fit compared with Poisson GLM (10270) and
# Linear LM (9873).
```

```
# Comparison to Poisson GLM: - Poisson coefficients are similar but
# AIC is much higher due to overdispersion. - Negative Binomial
# handles overdispersion, providing more reliable estimates.
```

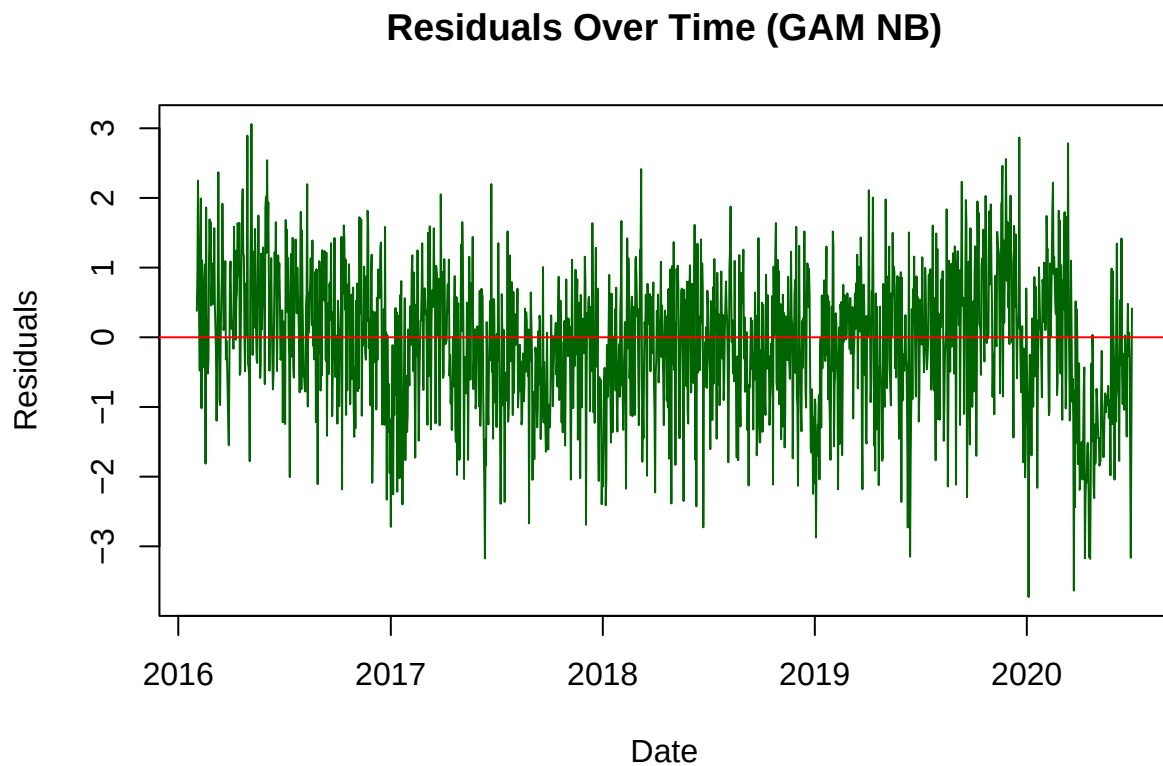
```
# GAM NegBin: - Slightly better AIC than NB GLM, showing that
# capturing nonlinear effects improves model fit. - Smooth terms
# (s(max_temp), s(EHF)) are significant, indicating nonlinear
# contributions. - Rain and min_temp have marginal or
# non-significant nonlinear effects.
```

```
# Conclusion: - GAM NegBin is the preferred model for predicting
# total daily accidents. - max_temp and EHF are the most influential
# predictors. - Rain has minimal predictive value in this dataset.
# - Nonlinear relationships (captured by GAM) slightly improve model
# performance over a strict linear NB GLM.
```

Q7.5

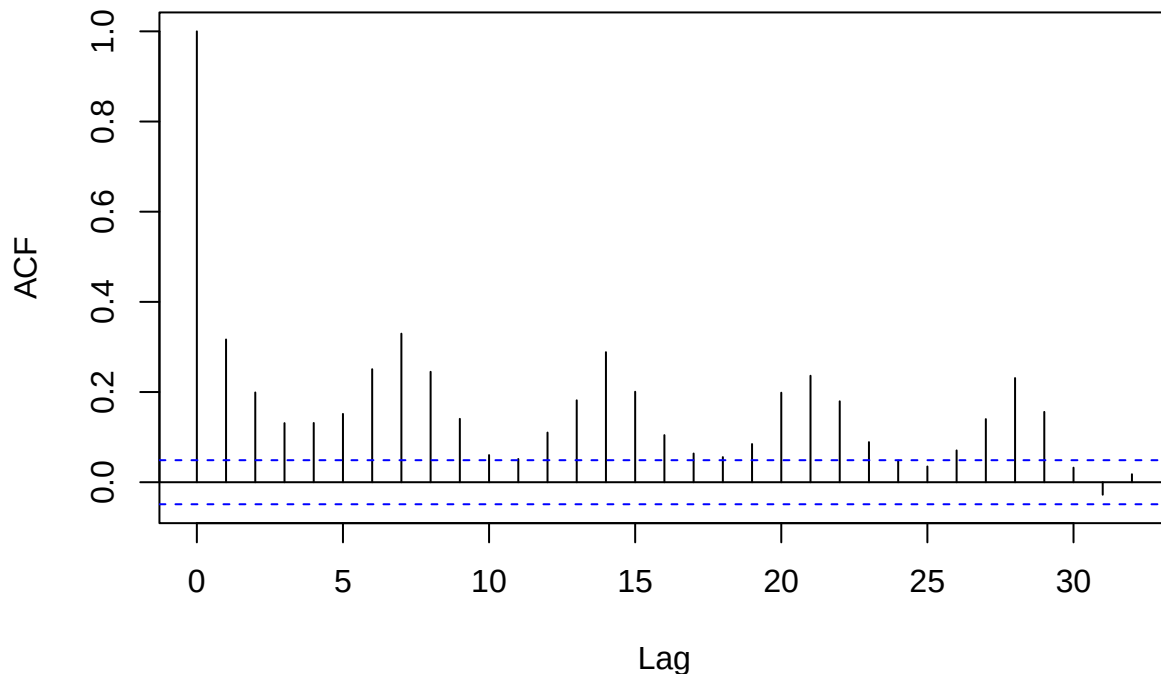
```
residuals_gam <- resid(fit_nb)

# Residuals over time
plot(model_df$DATE, residuals_gam, type = "l", col = "darkgreen", ylab = "Residuals",
      xlab = "Date", main = "Residuals Over Time (GAM NB)")
abline(h = 0, col = "red")
```



```
# Autocorrelation in residuals (GAM NB as best model)
acf(resid(fit_gam), main = "ACF of Residuals (GAM NB)")
```

ACF of Residuals (GAM NB)



The residual ACF plot shows significant autocorrelation at several lags beyond the 95% confidence bounds. This indicates that residuals are not independent, and the current model has not fully captured the time-dependent structure of the accident data. In particular, the presence of autocorrelation suggests that additional features (such as lagged accident counts or explicit seasonal terms) or a time-series-based model (e.g., GAMM or autoregressive models) may be required for a better fit.

Q7.6

```
# Compare GAM with and without EHF
fit_gam_noEHF <- gam(total_accidents ~ s(max_temp) + s(min_temp) + s(rain),
  data = model_df, family = nb())
```

```
AIC(fit_gam, fit_gam_noEHF)
```

```
##              df      AIC
## fit_gam      8.635400 9838.868
## fit_gam_noEHF 6.964215 9840.690
```

The predictor EHF improves model fit as shown by the AIC comparison: - GAM with EHF: AIC = 9838.868 - GAM without EHF: AIC = 9840.690\t Lower AIC for the model including EHF indicates a better trade-off between fit and complexity. Therefore, including EHF makes the model more parsimonious and informative for predicting

```
# total accidents.
```

Q7.7

```
# Create a lagged rain variable (previous day's rain)
model_df <- model_df %>%
  arrange(Date) %>%
  mutate(rain_lag1 = lag(rain, 1))

# GAM with temp_range included
fit_gam_extra <- gam(total_accidents ~ s(max_temp) + s(min_temp) + s(rain) +
  s(EHF) + s(rain_lag1), data = model_df, family = nb())

# Compare AIC
AIC(fit_gam, fit_gam_extra)
```

```
##              df      AIC
## fit_gam      8.635400 9838.868
## fit_gam_extra 9.819658 9835.398
```

```
model_df <- model_df %>%
  mutate(temp_range = max_temp - min_temp)

# GAM with temp_range included
fit_gam_extra <- gam(total_accidents ~ s(max_temp) + s(min_temp) + s(rain) +
  s(EHF) + s(temp_range) + s(rain_lag1), data = model_df, family = nb())

# Compare AIC
AIC(fit_gam, fit_gam_extra)
```

```
##              df      AIC
## fit_gam      8.63540 9838.868
## fit_gam_extra 14.99185 9824.150
```

Q7.7: Analysis of EHF and additional weather features EHF remains a useful predictor for road traffic accidents, as the GAM including EHF shows a lower AIC compared to models without it (GAM AIC = 9838.868). To capture delayed effects of weather, a lagged rain variable (rain amount from the previous day) was added. Including lagged rain in the GAM reduced the AIC slightly to 9835.398, indicating that the previous day's rainfall has a small but detectable impact on accident counts. Further, a temperature range variable (max_temp - min_temp) was introduced to account for daily temperature variability. Adding temp_range led to a further reduction in AIC to 9824.150, suggesting that days with larger temperature fluctuations are associated with higher accident counts. Interpretation: - EHF continues to improve model fit. - Lagged rain provides additional predictive value beyond same-day rainfall. - Temperature range is an informative feature, capturing variability effects on accidents. This demonstrates that incorporating lagged or derived weather variables can enhance the predictive accuracy of road traffic accident models.

Q8: Reflection

Q8.1:

*# Q8.1: Additional data to improve the model - Traffic volume and
type (cars, trucks) to account for exposure risk - Road
characteristics (type, lanes, speed limits, intersections,
lighting) - Temporal factors (day of week, holidays, time of day) -
Extreme weather events (fog, wind, ice, flooding) Including these
features could improve predictive accuracy beyond historical
accident and basic weather data.*

Q8.2

*# The analysis has partially addressed the research objective of
modeling the relationship between weather conditions and road
traffic accidents. The models successfully demonstrated that: -
Maximum temperature and EHF are significant predictors of accident
counts. - GAM models provide better fit than linear models for
capturing nonlinear relationships. - Lagged weather effects
(previous day's rain) are more predictive than same-day conditions.
However, the low R-squared values (approximately 1.5-1.9%) indicate
that weather variables alone explain only a small fraction of
accident variability. This suggests that while weather effects are
statistically significant, other factors (traffic volume,
infrastructure, human behavior) play much larger roles in
determining accident occurrence. The research objective was met in
demonstrating weather impacts, but highlighted the need for more
comprehensive modeling approaches incorporating multiple data
sources.*

Q8.3:

*# When dealing with missing values in traffic accident data, several
robust methods should be considered: Multiple Imputation (MI) is
the gold standard for handling missing data. MI creates multiple
plausible datasets with imputed values, analyzes each separately,
and pools results to account for uncertainty in the imputed values.
This method is particularly effective for maintaining statistical
power and reducing bias compared to listwise deletion.
Regression-based imputation methods can predict missing values
using relationships with other observed variables. For weather
data, missing temperature values could be imputed using data from
nearby weather stations or temporal interpolation techniques.
Machine learning approaches like Random Forest imputation have
shown superior performance for complex datasets with nonlinear
relationships. Model-based methods such as Maximum Likelihood
Estimation can handle missing data directly within the modeling
framework without explicit imputation. For time series weather
data, techniques like forward/backward filling or seasonal
decomposition methods can be particularly appropriate. The choice
of method should depend on the missing data mechanism (MCAR, MAR,
or MNAR) and the proportion of missing values. Sensitivity*

```
# analyses should be conducted to evaluate the robustness of results
# across different imputation methods.
```

Q8.4

```
# Overfitting occurs when a model fits the training data too closely,
# capturing noise instead of the underlying pattern, leading to poor
# generalization on new data. To control overfitting when managing
# many explanatory variables: 1. Model selection: - Use information
# criteria like AIC, BIC (Bayesian Information Criterion), or
# corrected AIC (AICc) which penalize model complexity. - Prefer
# simpler models that balance goodness-of-fit and parsimony. 2.
# Cross-validation: - Employ K-fold cross-validation or bootstrapping
# to estimate model performance on unseen data. - Use validation
# results to guide variable selection and tuning. 3. Regularization
# techniques: - Apply penalized regression methods such as LASSO (L1
# penalty) or Ridge (L2 penalty), which shrink coefficients of less
# relevant predictors towards zero. - Elastic Net combines both
# penalties to allow better variable selection in presence of
# correlated predictors. These methods help choose a robust model
# size and prevent overfitting.
```

Referaences

```
# References (APA 7th Edition)
```

```
# Cawley, G. C., & Nicola. (2025). Preventing Over-Fitting during
# Model Selection via Bayesian Regularisation of the
# Hyper-Parameters. Journal of Machine Learning Research, 8(31),
# 841-861. https://jmlr.org/papers/v8/cawley07a.html
```

```
# Eastern Victoria Region. (2022). Vic.gov.au; Victorian Electoral
# Commission.
# https://www.vec.vic.gov.au/electoral-boundaries/state-regions/eastern-victoria-region
```

```
# Heymans, M. W., & Twisk, J. (2022). Handling Missing Data in
# Clinical Research. Journal of Clinical Epidemiology, 151, 185-188.
# https://doi.org/10.1016/j.jclinepi.2022.08.016
```

```
# Kang, H. (2013). The Prevention and Handling of the Missing Data.
# Korean Journal of Anesthesiology, 64(5), 402-406.
# https://doi.org/10.4097/kjae.2013.64.5.402
```

```
# Király, B., & Hangya, B. (2022). Navigating the Statistical
# Minefield of Model Selection and Clustering in Neuroscience.
# Eneuro, 9(4), ENEURO.0066-22.2022.
# https://doi.org/10.1523/eneuro.0066-22.2022
```

```
# North and North West Metropolitan Melbourne - Regional Media
# Connect. (2016, June 20). Regional Media Connect.
# https://www.regionalmediaconnect.com.au/north-and-north-west-metropolitan-melbourne/
```

Northern Victoria Region. (n.d.). Victorian Electoral Commission.
 # <https://www.vec.vic.gov.au/electoral-boundaries/state-regions/northern-victoria-region>

Portet, S. (2020). A primer on model selection using the Akaike
 # Information Criterion. *Infectious Disease Modelling*, 5, 111-128.
 # <https://doi.org/10.1016/j.idm.2019.12.010>

Reported road casualties in Great Britain: notes, definitions,
 # symbols and conventions. (n.d.). GOV.UK.
 # <https://www.gov.uk/government/publications/road-accidents-and-safety-statistics-notes-and-definitions>

Solomon, N., Lokhnygina, Y., & Halabi, S. (2020). Comparison of
 # regression imputation methods of baseline covariates that predict
 # survival outcomes. *Journal of Clinical and Translational Science*,
 # 5(1). <https://doi.org/10.1017/cts.2020.533>

South-Eastern Metropolitan Region. (2025). Vic.gov.au.
 # <https://www.vec.vic.gov.au/electoral-boundaries/state-regions/south-eastern-metropolitan-region>

The North East Region: Victoria. (n.d.). Department of Agriculture,
 # Fisheries and Forestry (DAFF).
 # <https://www.agriculture.gov.au/sites/default/files/sitecollectiondocuments/rfa/regions/vic-north-east>

The South West Region. (2025). Swclimatechange.com.au.
 # https://www.swclimatechange.com.au/cb_pages/sw_region.php

Western Victoria Region. (2022). Victorian Electoral Commission.
 # <https://www.vec.vic.gov.au/electoral-boundaries/state-regions/western-victoria-region>